

Exercícios - Aula 08: Subprogramas

Paradigmas de Linguagens de Programação

Atividade: para cada trecho, identificar a saída do programa e explicar de forma simples qual conceito da aula justifica o resultado.

Parte 1/3

1. Python - parâmetro padrão

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

Saída: [1]

[1, 2]

Explicação: A lista usada como valor padrão não é criada novamente em cada chamada. Ela é criada uma vez e depois reutilizada. Por isso, quando o número 2 é adicionado, a lista ainda possui o 1 da chamada anterior.

Conceito: Parâmetro com valor padrão e passagem por atribuição.

2. Java - array e variável inteira

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
  
int[] v = {5, 5};  
int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

Saída: 0 5

Explicação: O método consegue alterar a primeira posição do array, porque a cópia da referência ainda aponta para o mesmo objeto. Já o valor de n é apenas uma cópia do inteiro, então alterar n dentro do método não muda o n original.

Conceito: Java passa os parâmetros por valor, inclusive as referências de objetos.

Parte 2/3

3. Python - lambdas dentro do laço

```
fs = [lambda: i for i in range(3)]  
print([f() for f in fs])
```

Saída: [2, 2, 2]

Explicação: As três funções lambda usam a mesma variável i. Elas não guardam uma cópia do valor em cada volta. Quando são executadas, o laço já terminou e i vale 2. Por isso as três retornam 2.

Conceito: Fechamentos (closures) e late binding.

4. C - variável local static

```
int contador(void) {
    static int n = 0;
    return ++n;
}

// em main:
contador();
contador();
printf("%d\n", contador());
```

Saída: 3

Explicação: Como n foi declarada como static, ela mantém seu valor entre uma chamada e outra. A primeira chamada deixa n = 1, a segunda deixa n = 2 e a terceira incrementa para 3, que é o valor mostrado.

Conceito: Variável local estática.

Parte 3/3

5. Rust - posse do vetor

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}

let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?}", v, d);
```

Saída: O programa não compila. O compilador gera o erro E0382.

Explicação: Ao chamar dobra(v), o vetor v é movido para a função, que passa a ter a posse dele. Depois disso, o v original não pode mais ser usado no println. Para continuar usando o vetor, seria necessário passar uma referência, como &v.;

Conceito: Posse (ownership/move) e passagem de parâmetros em Rust.

6. Python - variável global e local

```
total = 0

def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

Saída: O programa gera UnboundLocalError.

Explicação: Como existe uma atribuição para total dentro da função, o Python considera total uma variável local naquele escopo. Porém, na expressão total + x, ele tenta ler essa variável antes de ela receber um valor local. Uma solução seria usar global total, embora seja melhor receber o valor como parâmetro e retornar o resultado.

Conceito: Ambiente de referenciamento local e escopo.

Conclusão: Os exercícios mostram que a forma como parâmetros, referências, variáveis locais e fechamentos funcionam muda de uma linguagem para outra. Entender essas diferenças ajuda a prever o resultado do código e evitar alterações inesperadas nos dados.